



FACULTY OF ENGINEERING

**BACHELORS OF ENGINEERING IN MECHTRONICS AND BIOMEDICAL
ENGINEERING**

YEAR ONE: SEMESTER TWO (2026)

TEMB 1202: COMPUTING 2

LECTURER: MR. WAYNE STEPHEN OKELLO

TOPIC: FARM PRODUCE SALES SYSTEM TECHNICAL REPORT

GROUP B EVENING

NAME	REGISTRATION NUMBER
OKELLO ELVIS BECHKAM	25/U/BIE/01412/PE
OKIDI EMMANUEL OPIRO	25/U/BIE/01413/PE
SSENYANGE SIMON PETER	25/U/BIE/11579/PE
KASOZI JOASHIN SAMMY	25/U/BIE/01379/PE
BASAIJA PATRICIA	25/U/BIE/01362/PE

TABLE OF CONTENTS

ABSTRACT:

- 1. Introduction**
- 2. Problem statement and objectives**
- 3. System features**
 - 3.1 Produce Registration
 - 3.2 Search & Display
 - 3.3 Update & Delete
 - 3.4 Sell Produce
 - 3.5 Sales Summary & Reporting
- 4. Program Design & Structure**
 - 4.1 Data Structures
 - 4.2 System Architecture
 - 4.3 Module Breakdown
- 5. Business Rules & Validation Logic**
- 6. Key Functions & Implementation**
 - 6.1 Authentication
 - 6.2 Produce Management
 - 6.3 Sales & Discount
 - 6.4 Reporting
- 7. Design decisions & Justifications**
- 8. Challenges encountered**
- 9. Testing & Validation**
- 10. Conclusion**

ABSTRACT:

This system manages up to 100 produce items, enforces business rules on sales, supports bulk-buyer discounts, flags perishable and low-stock items, and generates a comprehensive sales report identifying the highest selling produce.

1. Introduction

This report documents the design and implementation of a **Farm Produce Sales System**, developed as part of the TEMB 1202 Computing 2 Group Assignment.

The system is written entirely in the **C Programming language** and provides a complete menu-driven interface for managing farm produce inventory and recording sales transactions.

Many local farm-produce stores require practical software tools to manage their operations efficiently. This system addresses that need by offering a structured, validated, and user-friendly console application that handles the full lifecycle of produce management; from stock registration to sales reporting.

2. Problem Statement & Objectives

Farm produce stores mainly rely on manual record keeping, which leads to errors in stock management, missed discount opportunities, and spoilage of perishable goods due to lack of timely alerts. The C program built automates these processes while enforcing a defined set of business rules.

2.1 Objectives

- Allow authorized users to register new produce items with full details.
- Enabling searching for produce by name or ID.
- Support updating produce details and deleting records.
- Process sales transactions with quantity validation and discount logic.
- Flag perishable items that have been in stock too long.
- Alert staff when stock levels fall below a restocking threshold.
- Warn about loss-making sales where the selling price is below cost price.
- Generate a summary report showing total sales, revenue, and remaining stock.
- Identify the highest selling produce item.
- Require authentication before system access.

3. System Features

The system provides six core areas accessible through a main menu. Each area is implemented as one or more dedicated C functions, ensuring modularity and ease of maintenance.

Feature	Description	Rules Applied
1. Register Produce	Add new produce with name, category, cost price, selling price, quantity, perishability, and stock date.	Duplicate ID check; mandatory fields.
2. Search Produce	Find a record by produce ID or browse all registered items with a formatted display	Case-intensive name search
3. Update Produce	Modify any field of an existing produce record after confirming the item exists.	Preserves unchanged fields.
4. Delete Produce	Remove a produce record from the system with user confirmation.	Confirmation prompt; cannot delete if unsold quantity 0
5. Sell Produce	Record a sale; validates stock, applies bulk discount, updates quantity, and accumulates revenue	All 5 business rules enforced
6. Sales Summary	Display total units sold, total revenue, remaining stock, and the highest selling produce.	Sorted report; identifies top seller.

3.1 Produce Registration

When registering a new produce item, the user enters: a unique numeric ID, the produce name, category (fruit/ vegetable/grain/dairy/other), cost price, initial quantity in stock, whether the item is perishable(yes/no), and the date it was added to the stock. The system validates that the ID does not already exist and that prices and quantities are positive values before saving the record.

3.2 Search & Display

Users may search by ID (direct lookup) or by name (substring match). The system displays all matching records in a neatly formatted table showing all fields. A 'display all' option lists every registered produce item and automatically flags perishable items whose stock age exceeds seven days.

3.3 Update & Delete

The update function presents the current values of a record and allows the user to selectively change any field. Deletion prompts for confirmation before removing the record, preventing accidental data loss.

3.4 Sell Produce

This is the most business critical function. It enforces all five rules: insufficient-stock check, bulk-buyer discount (10% for quantities ≥ 10 units), perishability age warning, low-stock restocking alert (threshold: 5 units remaining), and loss-warning when selling price is below cost price. Successful sales update the quantity on hand and accumulate totals.

3.5 Sales Summary & Reporting

The summary screen displays each produce item with units sold and revenue generated, then prints overall totals and identifies the highest-selling produce by units sold. This gives management an instant snapshot of store performance.

4. Program Design & Structure

4.1 Data Structures

All produce data is stored in a single C struct called **Produce**. An array of up to 100 such structs act as the in-memory database for the current session.

```
17 //DATA STRUCTURE
18 typedef struct{
19     int id;
20     char name[50];
21     char category[20];
22     float costPrice;
23     float sellingPrice;
24     int quantity;
25     int quantitySold;
26     float totalRevenue;
27     int isPerishable;
28     char stockDate[12]; /*DD/MM/YYYY*/
29     int daysInStock;
30 } Produce;
31 Produce inventory[MAX_PRODUCE];
32 int produceCount = 0;
33
```

4.2 System Architecture

The program follows a layered architecture with a clear separation between the user interface layer (menus and input), the business-logic layer (validation and rules), and the data layer (the in-memory struct array).

Layer	Responsibility	Key Functions
UI/ Menu	Display menus, collect user input, format output	main(), displayMenu(), printProduceTable()
Business Logic	Enforce all five business rules, compute discounts and flags	sellProduce(), checkPerishable(), applyDiscount()
Data management	CRUD operations on the inventory array	addProduce(), searchProduce(), updateProduce(), deleteProduce()
Reporting	Aggregate totals, identify top seller, display summary	generateSalesReport(), findTopSeller()
Authentication	Verify username and password before system access	Authenticate()

4.3 Module Breakdown

Authentication module: Implements a simple username/password check. Credentials are hardcoded for demonstration purposes but can be extended to file-based or database storage.

Produce management module: Contains addProduce(), searchById(), searchByName(), updateProduce(), and deleteProduce(). All operations validate inputs and provide meaningful error messages.

Sales module: The sellProduce() function orchestrates the complete sales workflow:

stock check >> discount calculation>> perishability warning >> low-stock alert >> loss warning >> transaction commit.

Reporting module: generateSalesReport() iterates the inventory array, prints per-item statistics, accumulates totals, and calls findTopSeller() to identify the best-performing produce.

Utility module: Helper functions for input sanitisation, date parsing, day-difference calculation, and formatted console output (tables, separators, colour prompts where supported).

5. Business Rules & Validation Logic

The assignment specifies five mandatory business rules. The table below documents each rule, then trigger condition, the system response, and where it is implemented in the code.

Rule	Trigger	System Response
Stock enforcement	Requested qty > available stock	Sale blocked; error message displayed.
Bulk Discount	Qty sold >= 10units	10% discount applied to selling price
Perishability Flag	Perishable items & days in stock > 7	Warning displayed; sale may proceed
Restocking Alert	Remaining qty <5 after sale	Restocking warning displayed post-sale
Loss warning	Selling price < Cost price	Loss warning displayed; sale may proceed

Note: On **Bulk Discount**, the 10% discount is applied to then unit selling price before multiplying by quantity. The effective price and the discount amount are both displayed to then user at the point of sale for full transparency.

6. Key Functions & Implementation

6.1 Authentication

The authentication function is called once at program startup. It prompts for a username and password, giving the user three attempts before locking out. On success, the main menu is displayed; on failure the program exits gracefully.

```

70 //authenticate()
71
72 int authenticate() {
73     char* authenticate() {
74         const char* VALID_USER="admin";
75         const char* VALID_PASS="farm2026";
76         int attempts=0;
77
78         while(attempts<3){
79             printf("\n===== \n");
80             printf("FARM PRODUCE SALES SYSTEM v1.0 \n");
81             printf("===== \n");
82             printf("USERNAME: ");
83             scanf("%29s",&user);
84             printf("PASSWORD: ");
85             scanf("%29s",&pass);
86             clearBuffer();
87             if (strcmp(user,VALID_USER)==0&&
88                 strcmp(pass,VALID_PASS)==0){
89                 printf("\n[OK] WELCOME, %s\n",user);
90                 return 1;
91             }
92             attempts++;
93             printf("[X] WRONG CREDENTIALS. %d attempt(s) left.\n",3-attempts);
94         }
95         printf("[X] ACCOUNT LOCKED. Contact administrator.\n");
96         return 0;
97     }
98 }

```

6.2 Produce Management-addProduce()

Collects all required fields with input validation, checks for duplicate IDs, then appends the new Produce struct to the inventory array and increments the counter.

```

115 //addProduce()
116 void addProduce() {
117     if (produceCount>=MAX_PRODUCE) {
118         printf("[!] INVENTORY FULL. \n"); return;
119     }
120     Produce p;
121     printf("\n-----REGISTER NEW PRODUCE-----\n");
122     printf("PRODUCE ID: \n");
123     scanf("%d",&p.id);
124     clearBuffer();
125     for (int i=0; i<produceCount; i++){
126         if (inventory[i].id==p.id){
127             printf("[!] ID %d ALREADY EXISTS.\n", p.id);return;
128         }
129     }
130     printf("NAME: \n");
131     fgets(p.name, 50,stdin);
132     p.name[strcspn(p.name, "\n")]='\0';
133     printf("CATEGORY: \n");
134     fgets(p.category,20,stdin);
135     p.category[strcspn(p.category, "\n")]='\0';
136     printf("COST PRICE: \n");
137     scanf("%f",&p.costPrice);
138     printf("SELLING PRICE: \n");
139     scanf("%f",&p.sellingPrice);
140     printf("QUANTITY: \n");
141     scanf("%d",&p.quantity);
142     printf("PERISHABLE? (1=YES 0=NO): \n");
143     scanf("%d",&p.isPerishable);
144     printf("STOCK DATE (DD/MM/YYYY): \n");
145     scanf("%11s",&p.stockDate);
146     clearBuffer();
147     if (p.costPrice<=0 ||p.sellingPrice<=0 ||p.quantity<0){
148         printf("[!]INVALID PRICE OR QUANTITY. \n"); return;
149     }
150     p.quantitySold=0;
151     p.totalRevenue=0.0f;
152     p.daysInStock=computeDayInStock(p.stockDate);
153     inventory[produceCount++]=p;
154     printf("[+] '%s' REGISTERED SUCCESFULLY. \n",p.name);
155 }

```

6.3 Sales & Discounts-sellProduce()

The most complex function in the system. It applies all five business rules in sequence and commits the transaction only after all checks pass.

```

239 //sellProduce-all 5 business rules
240 void sellProduce(){
241     int id,qty;
242     printf("\n=====SELL PRODUCE=====\\n");
243     printf("PRODUCE ID: ");
244     scanf("%d",&id);
245     printf("QUANTITY: ");
246     scanf("%d",&qty);
247     clearBuffer();
248     int idx=findById(id);
249     if (idx==-1){printf("[!]NOT FOUND.\\n"); return;}
250     Produce*p=&inventory[idx];
251
252     if (qty>p->quantity){
253         printf("[X]INSUFFICIENT STOCK! AVAILABLE: %d\\n",p->quantity);
254         return;
255     }
256     if (p->isPerishable && p->daysInStock>PERISH_DAYS)
257         printf("[!]PERISHABILITY WARNING: %s HAS BEEN STOCKED %d DAYS!\\n",p->name,p->daysInStock);
258     if (p->sellingPrice<p->costPrice)
259         printf("[!]LOSS WARNING: SELLING PRICE IS BELOW COST PRICE!\\n");
260
261     float unitPrice=p->sellingPrice;
262     if (qty>=BULK_THRESHOLD){
263         float disc=unitPrice*BULK_DISCOUNT;
264         unitPrice-=disc;
265         printf("[*]BULK DISCOUNT (10%%): -%.2f UGX/unit.NEW UNIT PRICE: %.2f\\n",disc,unitPrice);
266     }
267
268     float total=unitPrice*(float)qty;
269     p->quantity-=qty;
270     p->quantitySold+=qty;
271     p->totalRevenue+=total;
272     printf("[+]SALE COMPLETE: %d x %s=%.2f UGX\\n",qty,p->name,total);
273     if (p->quantity<LOW_STOCK_LVL)
274         printf("[!]RESTOCK ALERT: Only %d unit(s) of '%s' left!\\n",p->quantity,p->name);

```

6.4 Reporting-generateSalesReport()

```

276 //generateSalesReport()
277 void generateSalesReport(){
278     printf("\n=====\\n");
279     printf("FARM PRODUCE SALES REPORT \\n");
280     printf("=====\\n");
281     printf("%-4s %-18s %14s %9s\\n","ID","NAME","SOLD","REVENUE(UGX)","STOCK");
282     printf("-----\\n");
283     float totalRev=0.0f;
284     int totalSold=0,topIdx=-1;
285     for (int i=0; i<produceCount;i++){
286         Produce*p=&inventory[i];
287         printf("%-4d %-18s %6d %14.2f %9d\\n",p->id,p->name,p->quantitySold,p->totalRevenue,p->quantity);
288         totalRev+=p->totalRevenue;
289         totalSold+=p->quantitySold;
290         if(topIdx==-1 || p->quantitySold>inventory[topIdx].quantitySold)
291             topIdx=i;
292     }
293     printf("-----\\n");
294     printf("TOTAL SOLD: %d units\\n",totalSold);
295     printf("TOTAL REVENUE: %.2f UGX\\n",totalRev);
296     if (topIdx!=-1 && inventory[topIdx].quantitySold>0)
297         printf("TOP SELLER:%s(%d units)\\n",inventory[topIdx].name,inventory[topIdx].quantitySold);
298     else
299         printf("TOP SELLER: NO SALES RECORDED YET.\\n");
300     printf("=====\\n");
301 }

```

7. Design Decisions & Justifications

Single-file Architecture: The entire system is contained in one **Source Code.c** file. For a project of this scope, a single file reduces compilation complexity and makes submission straightforward. The code is organized with clear section comments for readability.

Array-Based In-memory storage: A fixed-size array of structs was chosen over dynamic allocation or file I/O to keep the implementation within the C concepts taught in class (arrays, structs, functions). File persistence could be added in a future iteration using `fread/fwrite`.

Hardcoded Authentication Credentials: For this demonstration, credentials are stored as string literals. In a production system, passwords would be hashed and stored in a configuration file.

10% Bulk Discount Threshold at 10units: A threshold of 10 units and 10% rate were chosen as they represent common retail price. Both constants are defined as macros (`#define`) so they can be adjusted without modifying function code.

Day-counting for Perishability: Rather than using the system clock (which requires and increases complexity), the `daysInStock` is computed at registration time from the entered stock date. This keeps the code simple while demonstrating the concept correctly.

User confirmation on delete: A y/n prompt before deletion is a widely-used UX best practice that prevents unintentional data loss.

Modular Function Design: Each operation is encapsulated in its own function making the code easier to test, debug, and extend. The `main()` function acts purely as a dispatcher keeping it concise.

8. Challenges Encountered

Maintaining data integrity during delete: Deleting an element from the middle of an array requires shifting subsequent elements. This was handled by a simple loop that shifts elements one position to the left and decrements the count.

Date arithmetic in C: Calculating the number of days a produce item has been in stock without using external libraries required a simplified date-difference formula.

Balancing simplicity and features: The assignment requires several features that, in a production system, would benefit from file I/O, dynamic memory, and more sophisticated data structures. Implementing all features within the constraints of basic C (as taught in class) required careful scoping and prioritization.

Testing edge cases: Business rules interact with each other. Systematic test cases were designed to verify that all warnings appear correctly and that only the stock-insufficient rule blocks sale.

Input buffer management: Mixing `scanf()` and `fgets()` in C causes well-known newline issues. This was resolved by implementing a `clearBuffer()` helper that consumes the remaining newline after each numeric `scanf` call.

9. Testing & Validation

The system was tested against a structured set of test cases covering normal flows, boundary conditions, and all five business rules. The table below summarizes the key test scenarios:

Test ID	Scenario	Expected result	Pass?
T-01	Valid login with correct credentials	System grants access	Pass
T-02	Three failed login attempts	Program exits after 3 rd failure	Pass
T-03	Register produce with duplicate ID	Error; ID already exists	Pass
T-04	Sell more than available stock	Sale blocked; error shown	Pass
T-05	Sell 10+ units	10% discount applied and shown	Pass
T-06	Sell perishable item > 7days old	Perishability warning shown	Pass
T-07	Sale leaves < 5 units remaining	Restock alert shown after sale	Pass
T-08	Selling price<cost price	Loss warning shown	Pass
T-09	Sales report with multiple products sold	Correct totals and top seller identified	Pass
T10	Delete with confirmation 'n'	Record preserved	Pass

10. Conclusion

The Farm Produce Sales System provides a complete menu-driven C program that allows authorized users to manage produce inventory through the full lifecycle from registration to sales reporting.

The program demonstrates a sound understanding of core C programming concepts including structs, arrays, functions, pointers, string handling, input validation, conditional logic, and loops. The modular design ensures that each feature can be independently extended or modified without affecting other parts of the system.

Some future enhancements could include file-based persistence (saving and loading the inventory between sessions), a graphical user interface, dynamic memory allocation to remove the 100-item cap, and more robust date arithmetic using the standard library.